

Clase 3: Visualización de Datos

Análisis de Datos y Estadística Computacional

Prof. Gabriel Duarte Zúñiga

Universidad Adolfo Ibáñez | DIAPP-2026

16 de junio de 2026

¡Bienvenid@s a la Clase 3!

Objetivo Clase 3

Esta clase tiene como objetivo aprender los principios fundamentales de la **visualización de datos** y aplicarlos mediante el paquete `ggplot2` de R ([Wickham 2015](#)).

Como objetivos específicos se busca:

- ▶ Comprender la importancia de una buena visualización y los errores más comunes a evitar.
- ▶ Construir visualizaciones con la **gramática de gráficos** (*grammar of graphics*): datos, estéticas, geometrías, escalas y temas.
- ▶ Aplicar distintos tipos de gráficos según el tipo de variable: `geom_bar()`, `geom_histogram()`, `geom_col()`, `geom_line()`, `geom_point()`, entre otros.
- ▶ Controlar los elementos de un gráfico: etiquetas, escalas y temas.
- ▶ Conocer extensiones: `gghighlight`, heatmaps y mapas geográficos.

La importancia de una buena visualización

¿Por qué visualizar?

El mayor valor de una imagen está en que nos obliga a notar algo que nunca esperamos ver ([Tukey 1977](#)).

1. **Explorar:** entender los datos para guiar análisis posteriores. Una visualización exploratoria es una forma de pensar y de usar gráficos como herramientas cognitivas para descubrir patrones (Wilke, 2019). → *Análisis exploratorio*
2. **Comunicar:** contar una historia con propósito, simplificando lo complejo y guiando la atención del público hacia el mensaje clave. → *Análisis aclaratorio*

Comunicar: foco en las perlas, no en las ostras



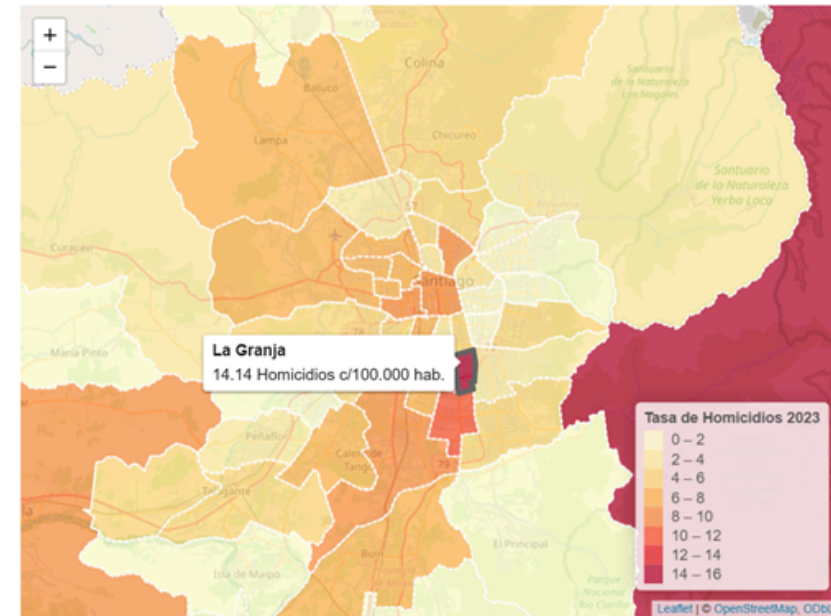
```
labels <- sprintf(
  "<strong>%s</strong><br/>%g Poblacion",
  comunas_santiago_leaflet$nombre_comuna, comunas_santiago_leaflet$poblacion
) %>% lapply(htmltools::HTML)

m %>% addPolygons(
  fillColor = ~pal(poblacion),
  weight = 2,
  opacity = 1,
  color = "white",
  dashArray = "3",
  fillOpacity = 0.7,
  highlightOptions = highlightOptions(
    weight = 5,
    color = "#666",
    dashArray = "",
    fillOpacity = 0.7,
    bringToFront = TRUE),
  label = labels,
  labelOptions = labelOptions(
    style = list("font-weight" = "normal", padding = "3px 8px"),
    textsize = "15px",
    direction = "auto")) |>
addLegend(pal = pal, values = ~poblacion, opacity = 0.7, title = NULL,
position = "bottomright")
```

Comunicar: foco en las perlas, no en las ostras

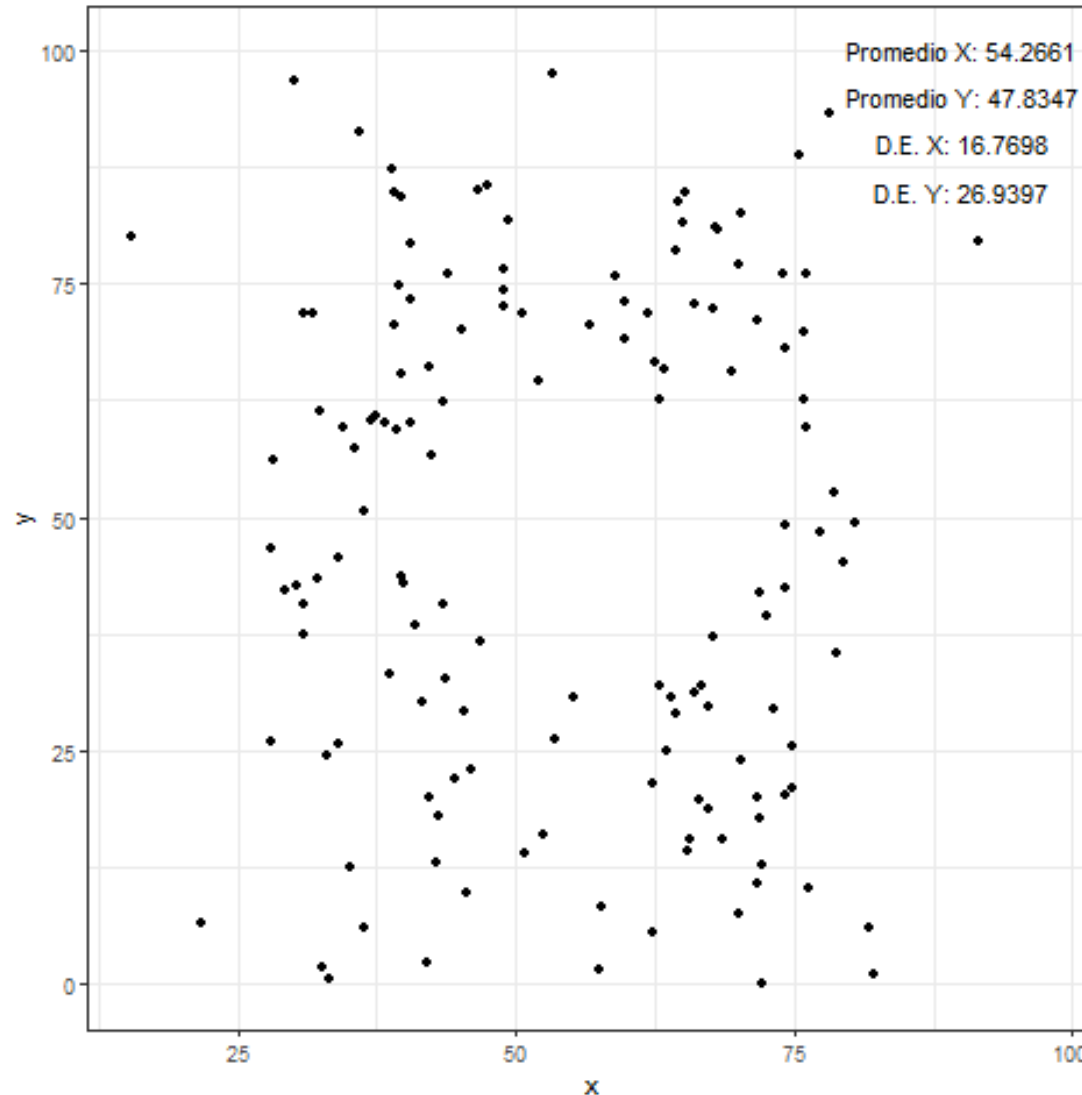


Tasa de Homicidios 2023



La importancia de entender los datos

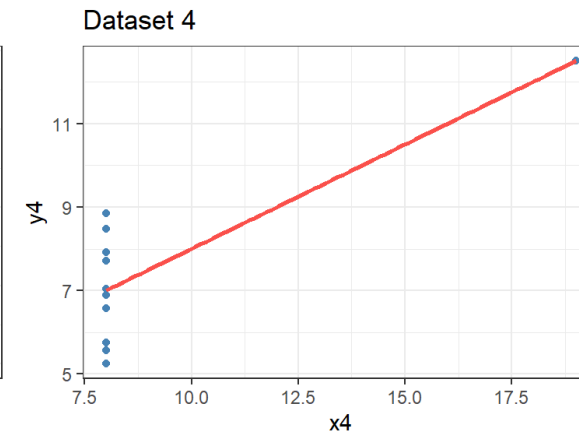
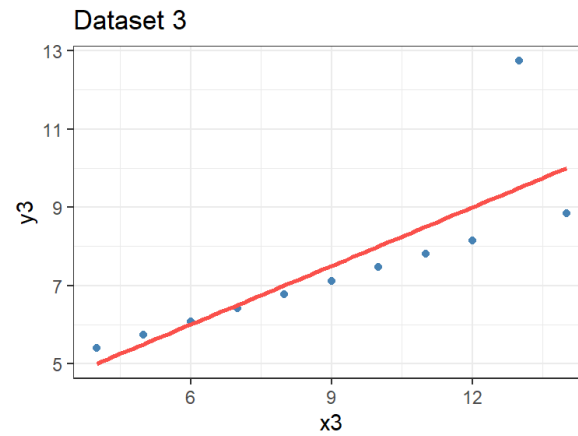
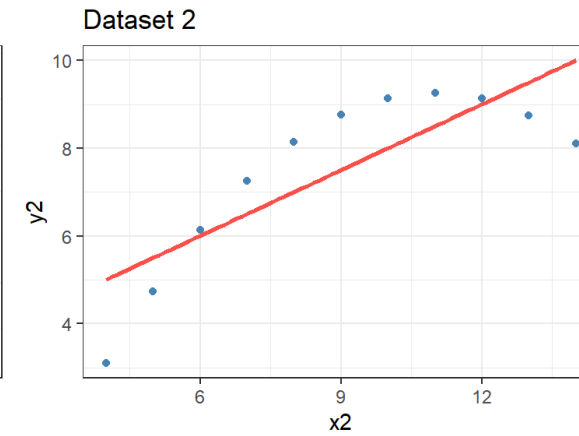
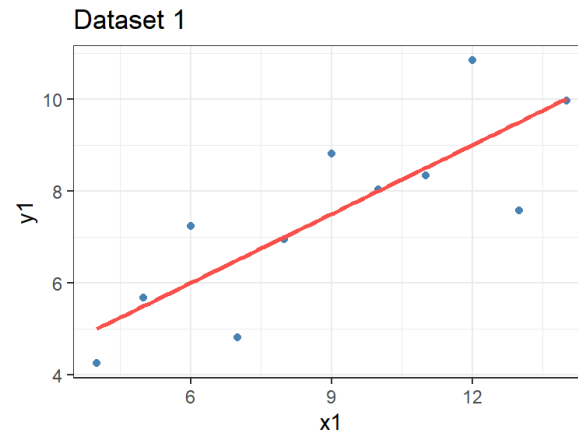
El Cuarteto de Anscombe (1973) es un conjunto de cuatro datasets con **estadísticas descriptivas idénticas** (media, varianza, correlación) que al graficarse revelan patrones completamente distintos.



El cuarteto de Anscombe

Gráficos

Datos



Lección principal



Mensaje principal

¡Confiar únicamente en medidas estadísticas resumidas sin una visualización adecuada puede conducir a conclusiones erróneas o incompletas!

Por eso **es importante visualizar los datos antes de empezar a modelar.**

Lo que deberíamos buscar

- ▶ Mostrar los datos y no mentir con estos.
 - Contar una historia (¿una relación? ¿causalidad? ¿un patrón? ¿un quiebre?).
 - Transmitir y convencer. No hacer *cherry picking*.
- ▶ Minimizar aspectos innecesarios que desvían la atención del público.
- ▶ Las visualizaciones deben complementar el texto y tener suficiente información para “sobrevivir por sí mismas”.

Atributos preatentivos

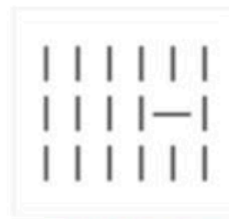
- ▶ El cerebro solo puede procesar un cierto número de atributos de forma instantánea (*pre-attentive attributes*). Los principales son: **forma**, **posición**, **color** y **tamaño**.
- ▶ Queremos buscar la variación justa en estos atributos para enfocarnos en lo que importa.



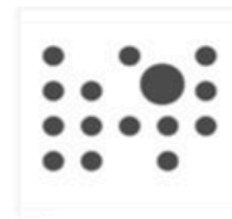
Largo



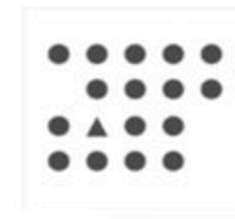
Ancho



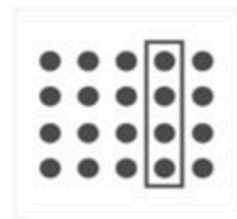
Orientación



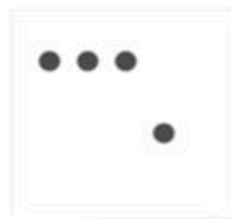
Tamaño



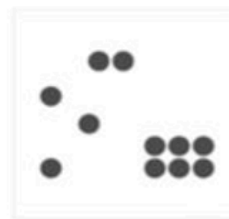
Forma



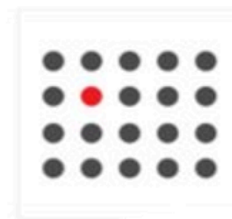
Demarcación



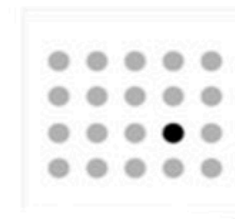
Posición



Agrupamiento



Tonalidad
de Color



Intensidad
de Color

([NUSSBAUMER KNAFLIC 2025](#))

¿Cuántos 3 hay?

756395068473
658663037576
860372658602
846589107830

([NUSSBAUMER KNAFLIC 2025](#))

¿Y ahora?

756**3**95068473
65866**3**0**3**7576
860**3**72658602
8465891078**3**0

([NUSSBAUMER KNAFLIC 2025](#))

Error #1: Violar el principio de proporcionalidad

The principle of proportional ink

Los tamaños de las áreas sombreadas en una visualización deben ser proporcionales a los valores de datos que representan.

En particular, este principio **no se cumple** en los gráficos de barra cuyo **eje y no parte en 0**.

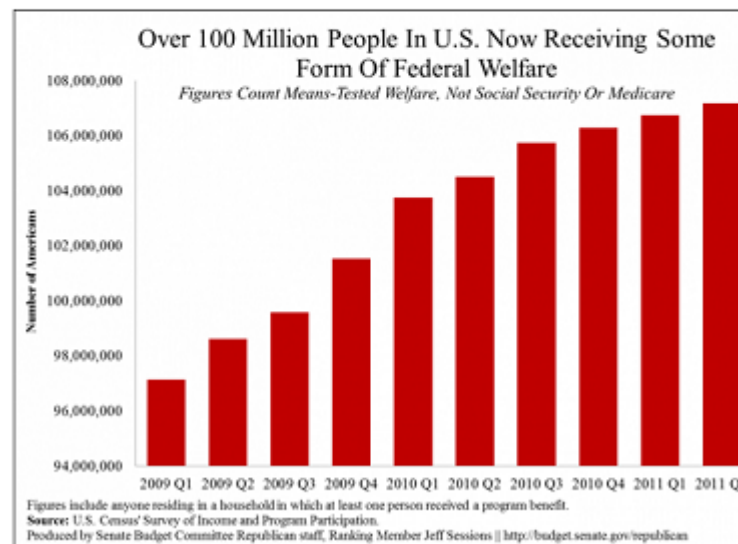
THE BLOG

Over 100 Million Now Receiving Federal Welfare

2:40 PM, AUG 8, 2012 • BY DANIEL HALPER 

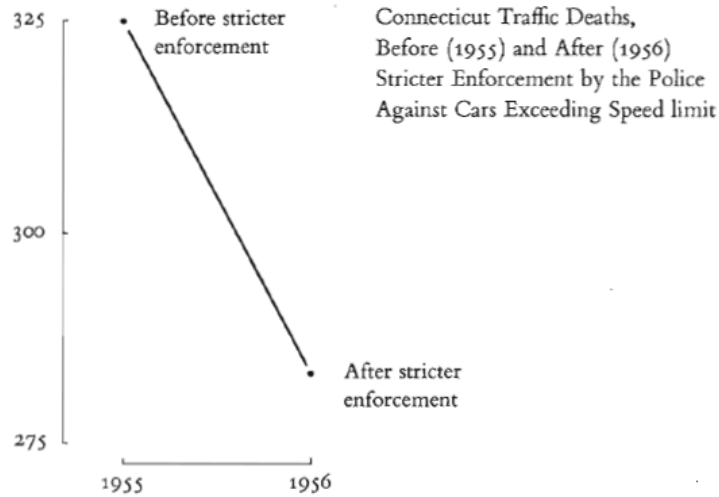
A new chart set to be released later today by the Republican side of the Senate Budget Committee details a startling statistic: "Over 100 Million People in U.S. Now Receiving Some Form Of Federal Welfare."



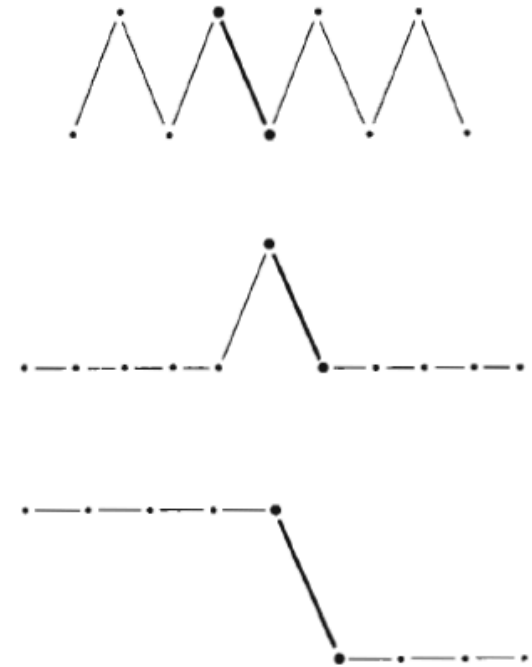
Misleading Graphs

Error #2: No entregar contexto

- ▶ Cuando mostramos una evolución, es importante preguntarse: **¿comparado con qué?**

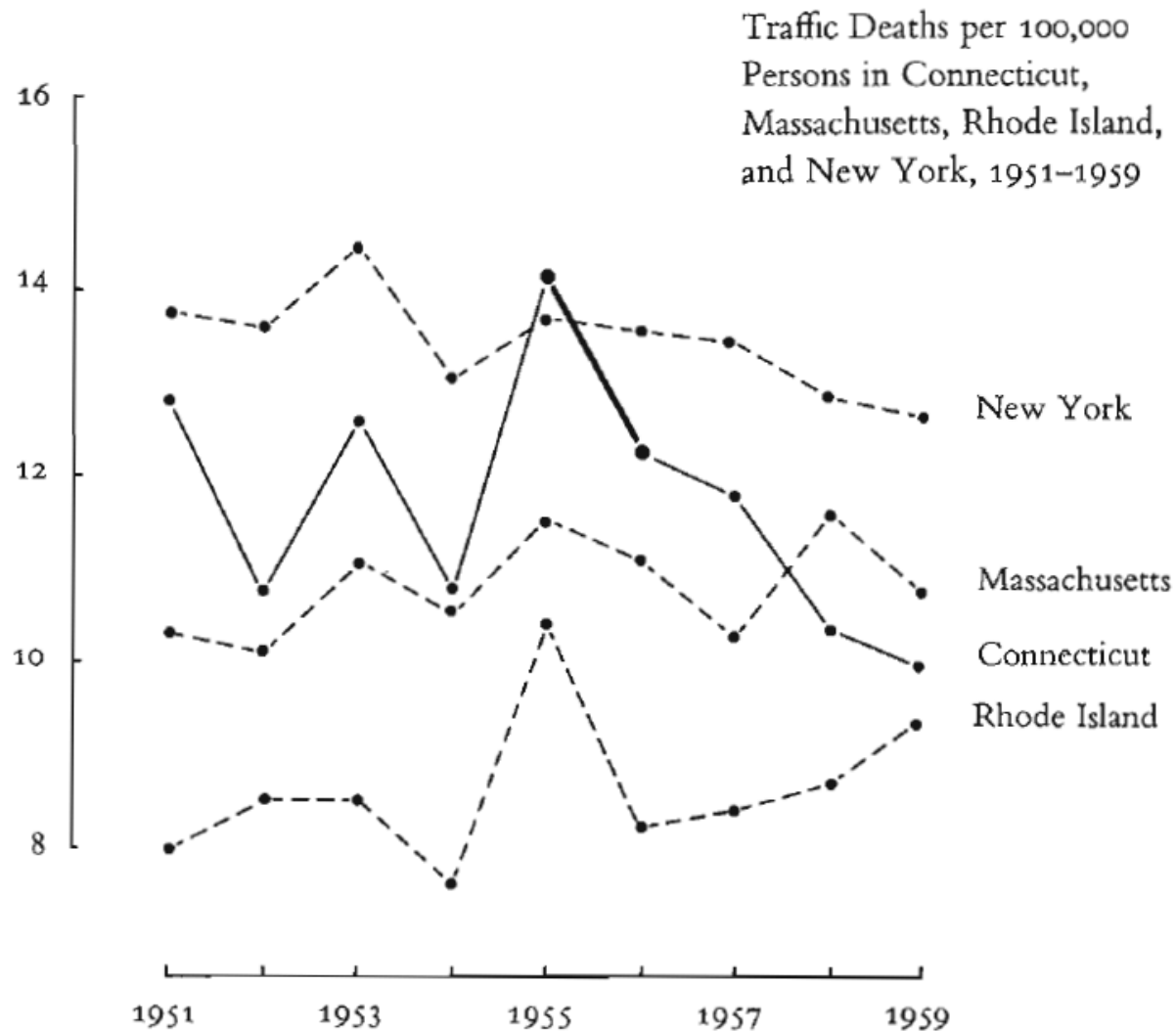


([Tufte 2001](#))



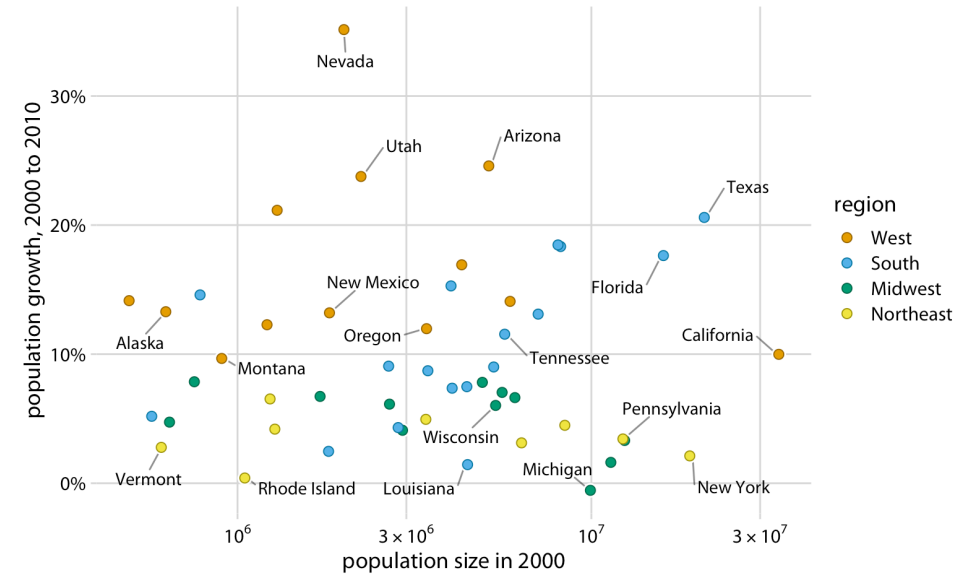
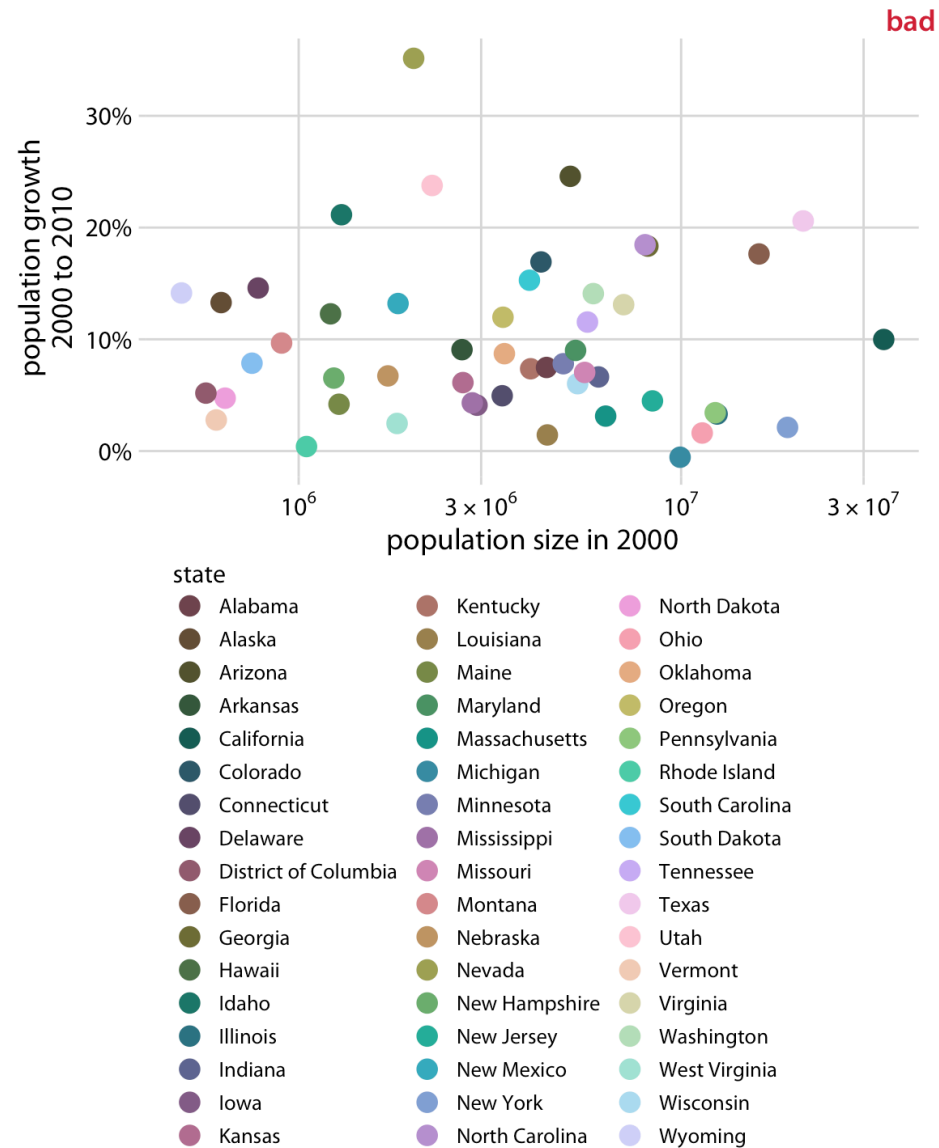
Posibles tendencias ([Tufte 2001](#))

Error #2: No entregar contexto



Donald T. Campbell and H. Laurence Ross, "The Connecticut Crackdown on Speeding: Time Series Data in Quasi-Experimental Analysis," in Edward R. Tufte, ed., *The Quantitative Analysis of Social Problems* (Reading, Mass., 1970), 110–125.

Error #3: Mostrar demasiada información



Reducir las categorías de color y etiquetar directamente mejora significativamente la legibilidad ([Wilke 2019](#)).

Construyendo visualizaciones con ggplot2

ggplot2

¿Qué es?

- ▶ Un paquete de R para producir gráficos estadísticos basado en la *grammar of graphics*, que permite componer gráficas combinando componentes **independientes**.

¿Cuál es su ventaja?

- ▶ Permite trabajar **iterativamente**, añadiendo *capas* al gráfico, haciendo más fácil la transición entre una idea a un gráfico real.

Grammar of Graphics

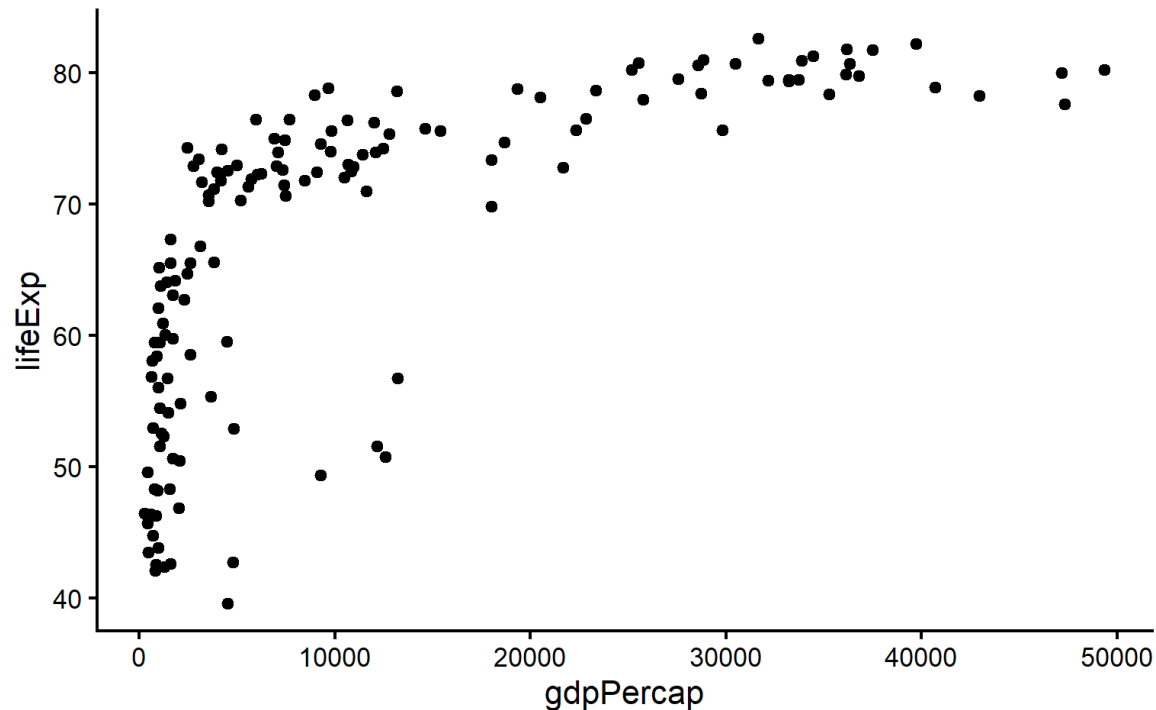


- ▶ **Data:** los datos a graficar.
- ▶ **Aesthetics** (**aes**): mapean variables a atributos visuales (posición x/y, color, forma, tamaño).
- ▶ **Geoms** (**geom_**): los objetos geométricos que se dibujan (puntos, líneas, barras).
- ▶ **Scales** (**scale_**): controlan los ejes y leyendas.
- ▶ **Facets** (**facet_**): dividen el gráfico por subgrupos.
- ▶ **Theme** (**theme_**): controlan los aspectos estéticos del fondo.

La estructura básica en 3 pasos

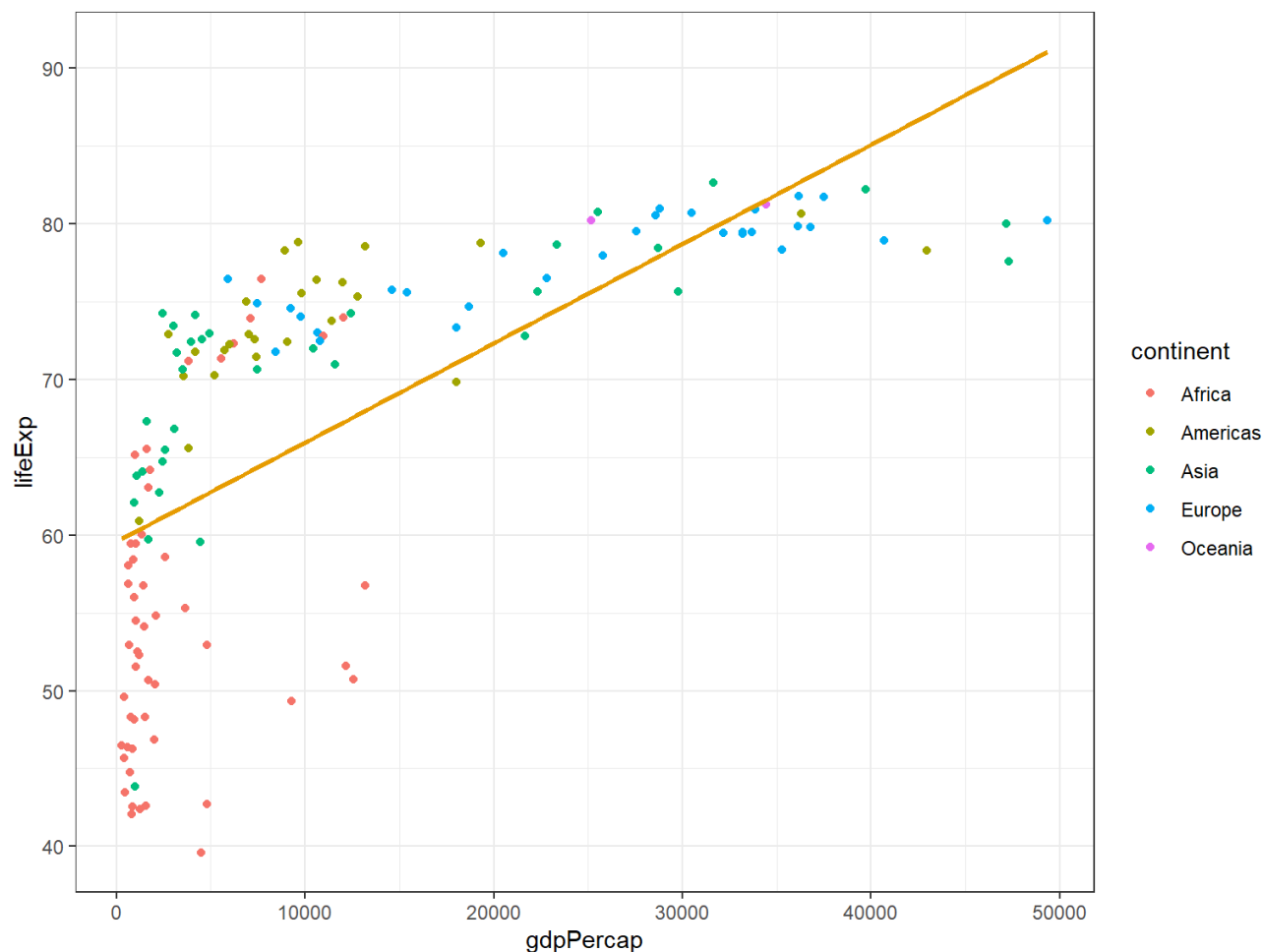
1. **Especificar los datos:** `ggplot(data = datos)`
2. **Especificar los aesthetics:** `aes(x = ..., y = ...)`
3. **Agregar un geom:** `+ geom_point()`

```
1 gap_2007 <- gapminder |> filter(year == 2007)
2
3 ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
4   geom_point()
```



Construyendo capa por capa

```
1 ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +  
2   geom_point(aes(color = continent)) +  
3   geom_smooth(se = FALSE, method = "lm", color = "#E69F00") +  
4   theme_bw()
```





Ejercicio 1: Histograma

Usando `gapminder` (año 2007), construya un histograma de la esperanza de vida (`lifeExp`). Rellene las barras de azul y use 10 intervalos.

R Code

[Start Over](#)

[Run Code](#)

```
1 library(gapminder)
2 library(ggplot2)
3 library(dplyr)
4
5 gap_2007 <- gapminder |> filter(year == 2007)
6
7 ggplot(gap_2007, aes(x = ____)) +
8   geom_histogram(fill = "____", color = "black", bins = ____ ) +
9   labs(x = "Esperanza de vida", y = "Frecuencia") +
10  theme_minimal()
```

Visualizaciones según el tipo de dato

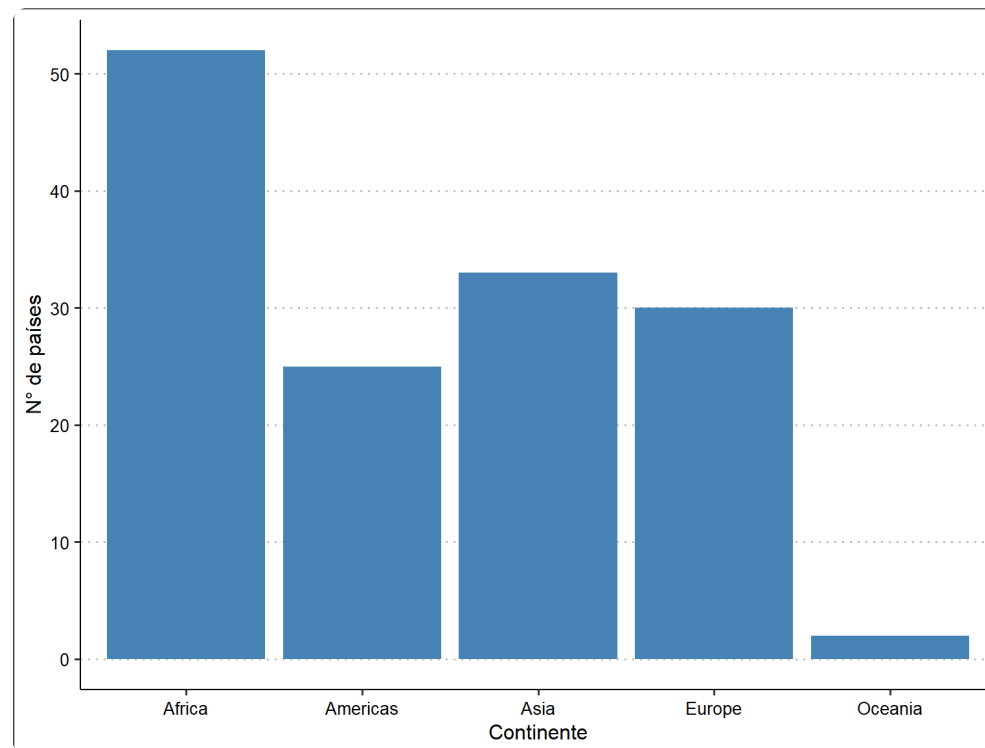
N° Variables	Tipo	Geom recomendado
1	Categórica	<code>geom_bar()</code>
1	Cuantitativa	<code>geom_histogram()</code> , <code>geom_density()</code>
2	Categórica + Categórica	<code>geom_bar(position = "dodge")</code>
2	Cuantitativa + Cuantitativa	<code>geom_point()</code> , <code>geom_line()</code>
2	Categórica + Cuantitativa	<code>geom_col()</code> , <code>geom_boxplot()</code>

Aplicaciones: tipos de **geom**

Barras para datos categóricos: `geom_bar()`

- `geom_bar()` **cuenta** las observaciones por categoría de manera automática, por lo que **no necesita** especificar en `aes()` el eje y:

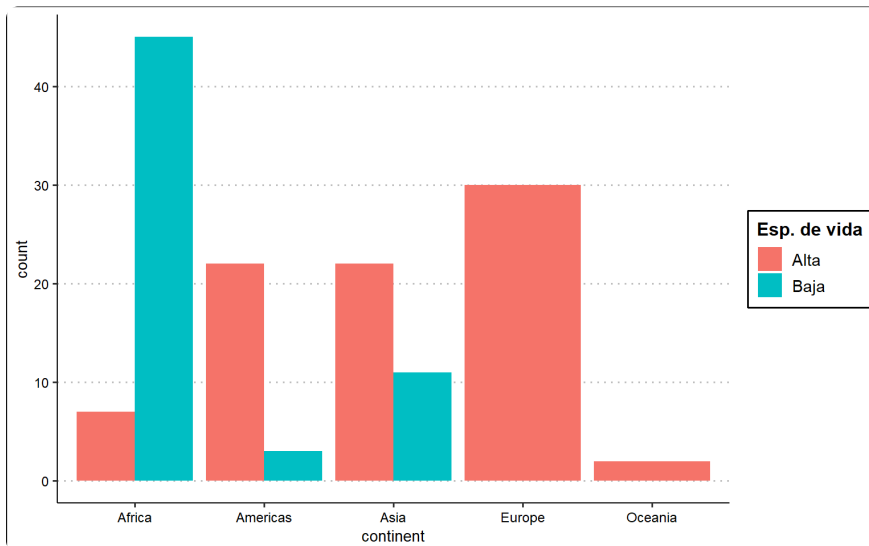
```
1 ggplot(gap_2007, aes(x = continent)) +  
2   geom_bar(fill = "steelblue") +  
3   labs(x = "Continente", y = "Nº de países") +  
4   theme_clean(base_size = 14)
```



Barras agrupadas y apiladas

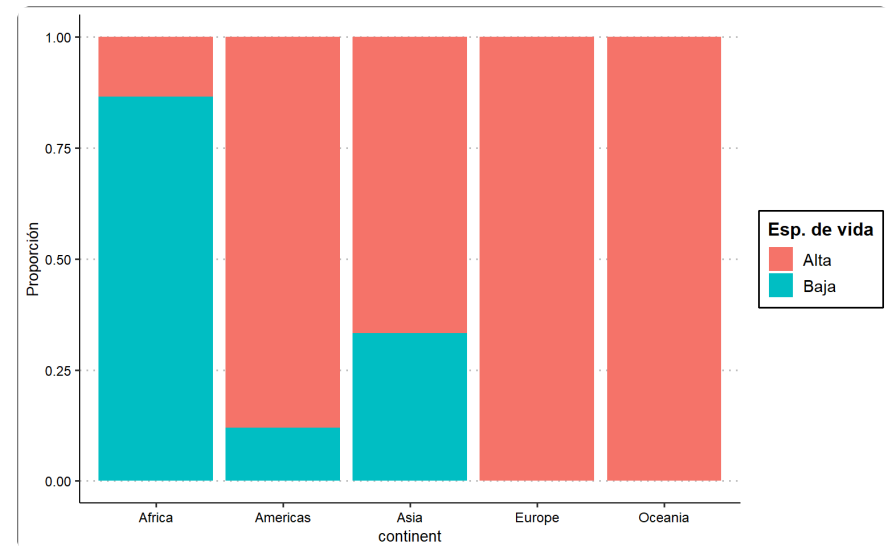
Con `position = "dodge"` dentro de `geom_bar()` podemos **agrupar las barras** (ponerlas juntas) en función de una segunda categoría indicada en `fill`:

```
1 ggplot(gap_2007, aes(x = continent,
2   fill = if_else(lifeExp > 70,
3     "Alta", "Baja"))) +
4   geom_bar(position = "dodge") +
5   labs(fill = "Esp. de vida") +
6   theme_clean(base_size = 12)
```



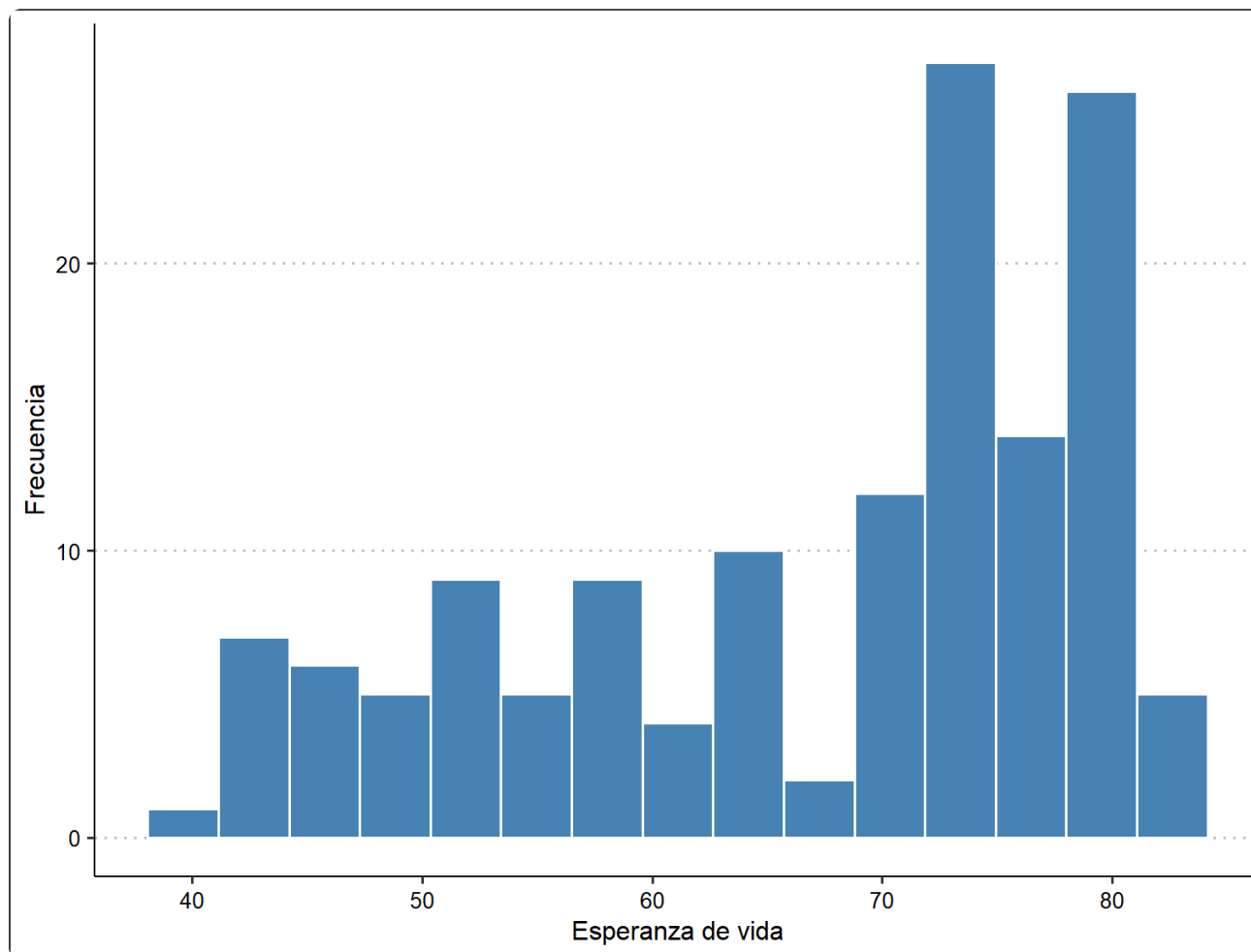
Con `position = "fill"` dentro de `geom_bar()` podemos **apilar las barras** (una encima de otra) en función de una segunda categoría indicada en `fill`:

```
1 ggplot(gap_2007, aes(x = continent,
2   fill = if_else(lifeExp > 70,
3     "Alta", "Baja"))) +
4   geom_bar(position = "fill") +
5   labs(y = "Proporción", fill = "Esp. de vi
6   theme_clean(base_size = 12)
```



Distribuciones: `geom_histogram()`

```
1 ggplot(gap_2007, aes(x = lifeExp)) +  
2   geom_histogram(fill = "steelblue", color = "white", bins = 15) +  
3   labs(x = "Esperanza de vida", y = "Frecuencia") +  
4   theme_clean(base_size = 14)
```



Distribuciones: `geom_density()` y `geom_boxplot()`



Pregunta N° 1

Queremos mostrar la **distribución** de la esperanza de vida en 2007, comparando visualmente **África vs. América** en el mismo gráfico.

¿Cuál es la combinación más adecuada?

```
geom_bar(fill = continent)
```

```
geom_histogram(fill = continent)
```

```
geom_density(fill = continent) + alpha
```

```
geom_boxplot(x = continent, y = lifeExp)
```

[Revisar](#)[Reiniciar](#)

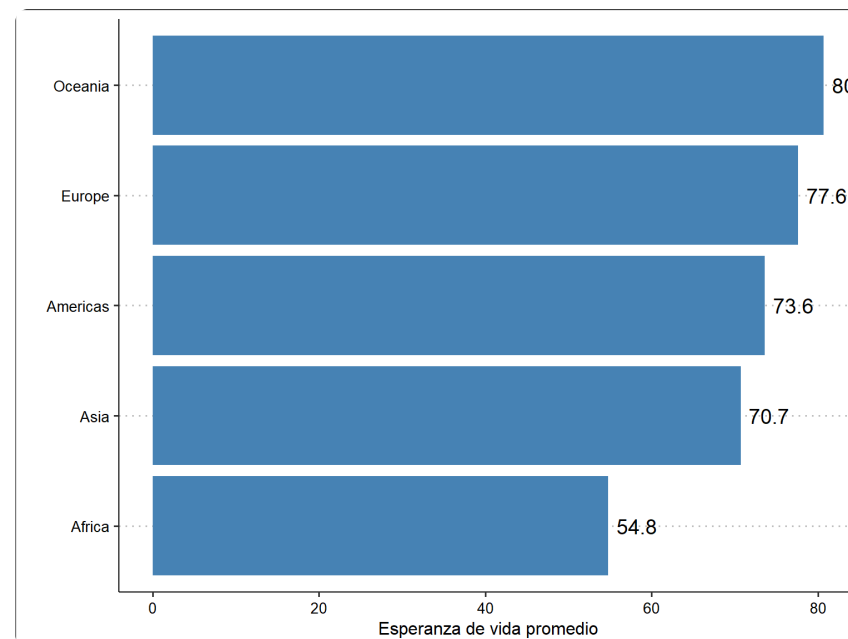
Columnas numéricas: `geom_col()`

- A diferencia de `geom_bar()` (que cuenta), `geom_col()` usa directamente el valor de `y` como altura de la barra:

```

1 gap_2007 |>
2   group_by(continent) |>
3   summarise(media_vida = mean(lifeExp)) |>
4   ggplot(aes(
5     x = reorder(continent, media_vida),
6     y = media_vida)
7   ) +
8   geom_col(fill = "steelblue") +
9   geom_text(aes(
10    label = round(media_vida, 1)),
11    hjust = -0.2
12  ) +
13  coord_flip() +
14  labs(x = "", y = "Esperanza de vida prom")
15  theme_clean(base_size = 14)

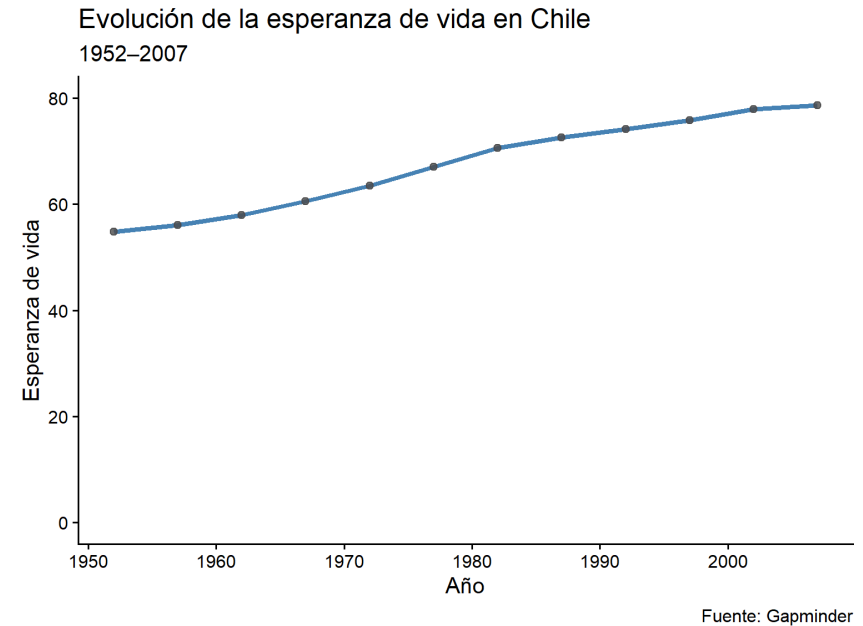
```



Series de tiempo: `geom_line()`

Con `geom_line()` podemos generar líneas, las cuales son muy útiles para mostrar evoluciones de indicadores a lo largo del **tiempo**:

```
1 gapminder |>
2   filter(country == "Chile") |>
3   ggplot(aes(x = year, y = lifeExp)) +
4   geom_line(linewidth = 1.5, color = "steelblue") +
5   geom_point(size = 2, color = "gray30", alpha = 0.5) +
6   scale_y_continuous(limits = c(0, 80)) +
7   labs(title = "Evolución de la esperanza
8         subtitle = "1952-2007",
9         x = "Año", y = "Esperanza de vida",
10        caption = "Fuente: Gapminder") +
11   theme_classic(base_size = 15)
```





Ejercicio 2: Gráficos de línea

Complete el código para graficar la evolución de la esperanza de vida **promedio por continente**, coloreando cada línea por continente:

R Code

[↺ Start Over](#)

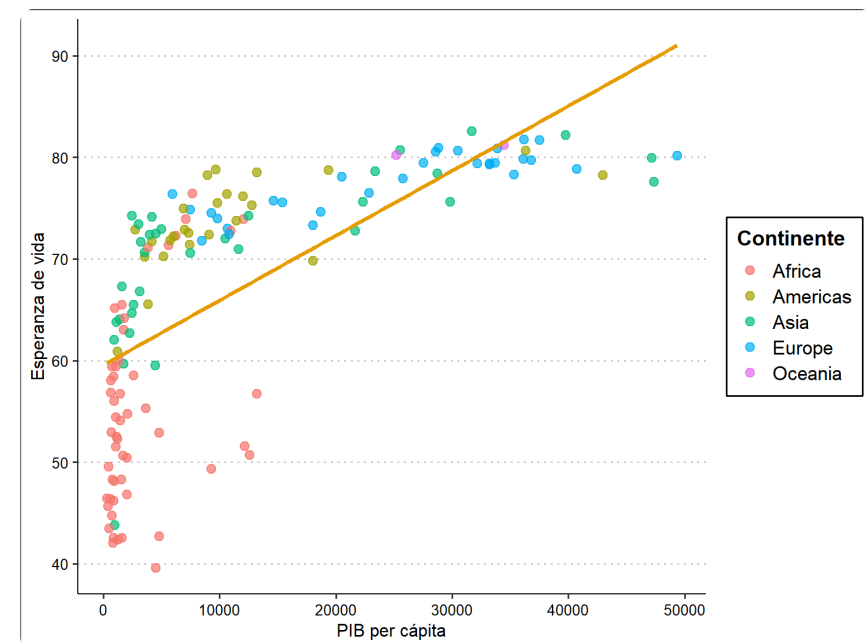
[▶ Run Code](#)

```
1 library(gapminder)
2 library(ggplot2)
3 library(dplyr)
4
5 gapminder |>
6   group_by(continent, year) |>
7   summarise(lifeExp = mean(lifeExp), .groups = "drop") |>
8   ggplot(aes(x = year, y = lifeExp, color = ____)) +
9   geom_line(linewidth = 1.5) +
10  geom_point(size = 2, alpha = 0.9) +
11  scale_y_continuous(limits = c(0, NA)) +
12  labs(title = "Esperanza de vida por continente",
13       x = "Año", y = "Esperanza de vida",
14       color = "Continente") +
15  theme_minimal(base_size = 14)
```

Scatterplot: `geom_point()`

Con `geom_point()` y dos variables cuantitativas en x e y podemos hacer un gráfico de dispersión o *scatterplot*, el cual es muy útil para mostrar la **relación entre dos variables numéricas**. Adicionalmente, con `geom_smooth()` podemos ajustar una recta o curva para entender el patrón que hay en los datos:

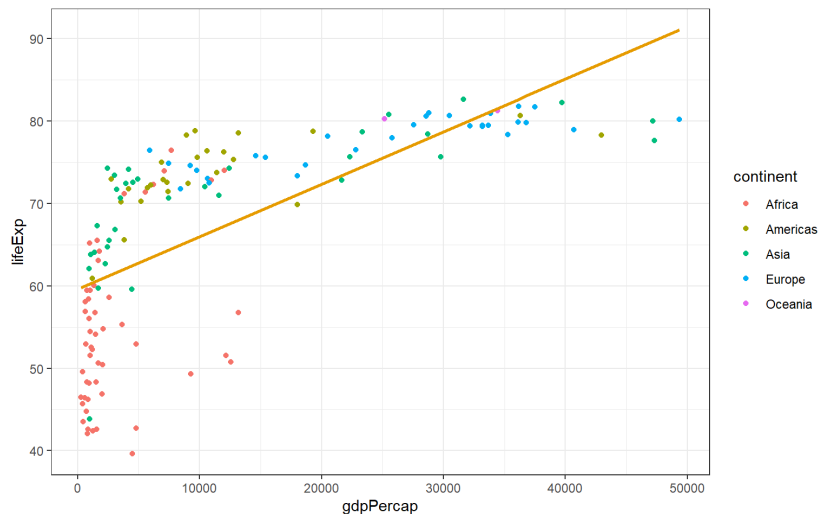
```
1 ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +
2   geom_point(aes(color = continent), #Color
3             alpha = 0.7, #Transparencia
4             size = 2.5) + #Tamaño de los
5   geom_smooth(se = FALSE, #Sin intervalo de confianza
6             method = "lm", #Recta ajustada
7             color = "#E69F00") + #Color
8   labs(x = "PIB per cápita", y = "Esperanza de vida") +
9   theme_clean(base_size = 14)
```



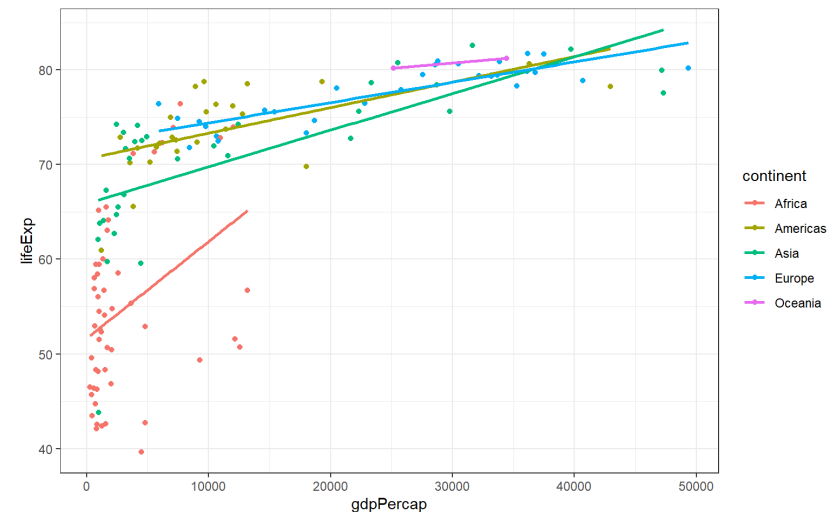
Entendiendo el nivel del `aes()`

En qué nivel del `aes` se fija el `color` determina **qué capas** lo heredan:

```
1 # Color SOLO en los puntos
2 ggplot(gap_2007,
3   aes(x = gdpPercap,
4     y = lifeExp)) +
5   geom_point(aes(color = continent)) +
6   geom_smooth(method = "lm",
7     se = FALSE, color = "#E69F00") +
8   theme_bw()
```



```
1 # Color en puntos + líneas
2 ggplot(gap_2007,
3   aes(x = gdpPercap,
4     y = lifeExp,
5     color = continent)) +
6   geom_point() +
7   geom_smooth(method = "lm",
8     se = FALSE) +
9   theme_bw()
```



Controlando elementos

Etiquetas: `labs()`

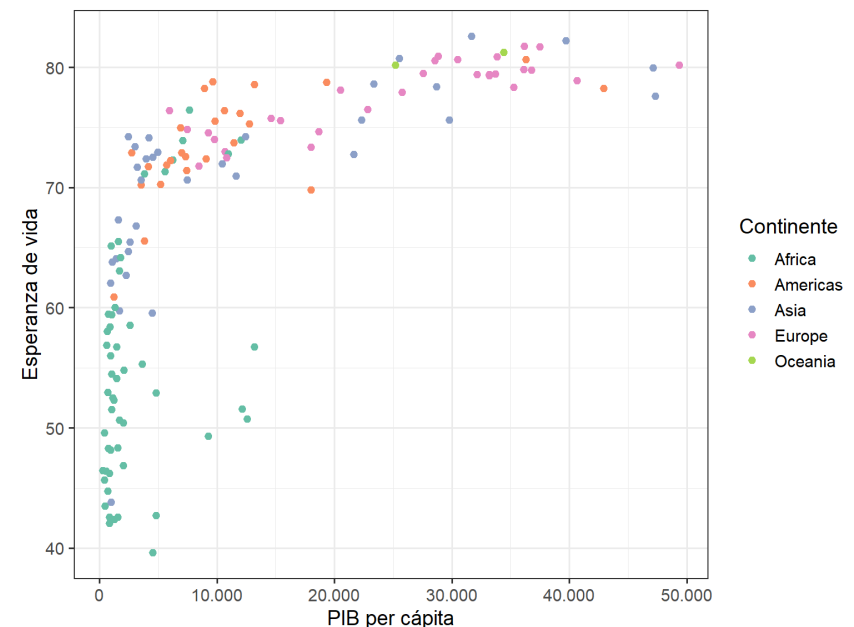
- `labs()` permite definir **todos** los textos del gráfico en un solo lugar:

```
1 ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp)) +  
2   geom_point() +  
3   labs(  
4     title      = "Relación entre PIB y esperanza de vida",  
5     subtitle   = "Año 2007",  
6     x          = "PIB per cápita (USD)",  
7     y          = "Esperanza de vida (años)",  
8     color      = "Continente",  
9     caption    = "Fuente: Gapminder"  
10  )
```

Escalas: `scale_*()`

Las funciones `scale_*()` controlan los **ejes y leyendas**. Por ejemplo, con `scale_x_continuous()` indicamos que el eje x es una variable continua y con sus argumentos de `labels` y `breaks` podemos especificar el formato en que se muestran las etiquetas y los quiebres o cantidad de etiquetas que aparezcan. Por último, si se ha fijado un `color` o `fill` dentro de un `aes()`, las funciones `scale_color_*()` o `scale_fill_*()` permiten usar paletas de colores predefinidas:

```
1 ggplot(gap_2007, aes(
2   x = gdpPercap,
3   y = lifeExp)
4 ) +
5   geom_point(aes(color = continent)) +
6   scale_x_continuous(
7     labels = label_number(
8       big.mark = ".",
9       decimal.mark = ",",
10    ),
11    breaks = seq(0, 50000, 10000)
12 ) +
13   scale_color_brewer(palette = "Set2") +
14   labs(x = "PIB per cápita",
15        y = "Esperanza de vida",
16        color = "Continente") +
17   theme_bw(base_size = 14)
```



Temas: `theme_*`()

Los temas controlan los aspectos **no relacionados con los datos**: fondo, grillas, fuentes, etc. Algunos temas populares:

- ▶ `theme_minimal()` — limpio, sin bordes
- ▶ `theme_clean()` (ggthemes) — minimalista
- ▶ `theme_classic()` — solo ejes, sin grilla
- ▶ `theme_economist()` (ggthemes) — estilo The Economist
- ▶ `theme_bw()` — fondo blanco con bordes



Tip

Pueden personalizar cualquier elemento con `theme()`. Por ejemplo, `theme(legend.position = "bottom")` mueve la leyenda abajo, y `theme(plot.title = element_text(face = "bold"))` pone el título en negrita.



Pregunta N° 2

Tenemos el siguiente gráfico y queremos que el eje x muestre los valores del PIB per cápita con **punto como separador de miles** (convención chilena). ¿Qué función usamos?

¿Cuál es la opción correcta?

```
labs(x = label_number(big.mark = "."))
```

```
scale_x_continuous(labels = label_number(big.mark = '.', decimal.mark = ','))
```

```
theme(axis.text.x = element_text(big.mark = '.'))
```

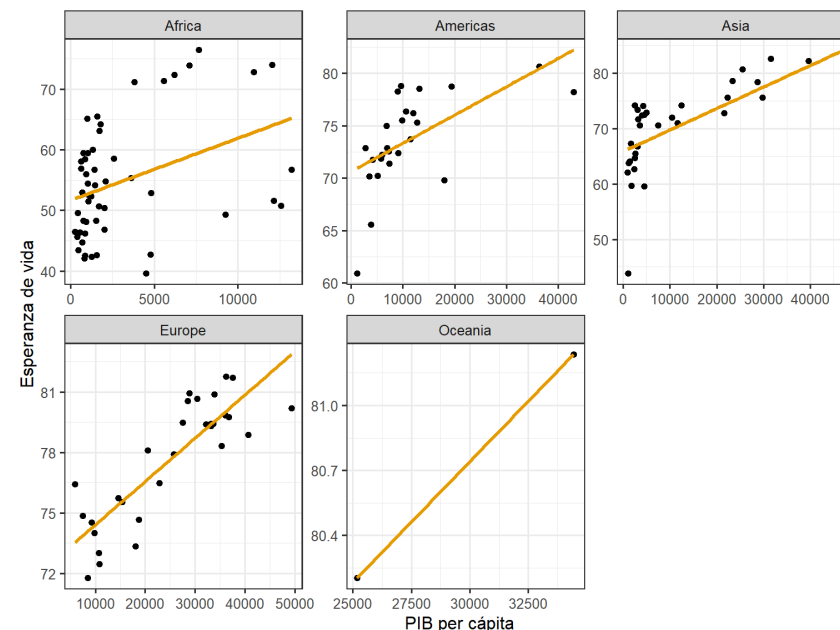
```
geom_text(aes(label = format(gdpPercap, big.mark = '.')))
```

[Revisar](#)[Reiniciar](#)

Subgráficos por grupo: `facet_wrap()`

La función `facet_wrap()` crea una cuadrícula de gráficos, uno por cada grupo de una variable, permitiendo comparar visualmente los datos entre categorías de forma rápida y ordenada. Con la función `vars` como primer argumento se pueden especificar las variables por las cuales se quieren generar las combinaciones de subgráficos. Con el argumento `scales` es posible dejar que la escala de los ejes x e y se ajusten automáticamente a partir de la distribución de datos en cada subgrupo:

```
1 ggplot(gap_2007,
2       aes(x = gdpPercap,
3           y = lifeExp)) +
4   geom_point() +
5   geom_smooth(se = FALSE,
6             method = "lm",
7             color = "#E69F00") +
8   facet_wrap(vars(continent),
9             scales = "free") +
10  labs(x = "PIB per cápita", y = "Esperanz
11  theme_bw(base_size = 12)
```



Exportar gráficos: `ggsave()`

- `ggsave()` guarda el **último gráfico** producido en un archivo:

```
1 ggsave("output/gapminder_continente.png",  
2       width  = 10,  
3       height = 10,  
4       dpi    = 300,  
5       bg     = "white")
```



Tip

`dpi = 300` asegura resolución de impresión. `bg = "white"` fuerza fondo blanco cuando el tema tiene fondo transparente (como `theme_minimal()`).



Ejercicio 3: Scatterplot y tendencia

Construya un scatterplot con PIB per cápita vs esperanza de vida, coloreado por continente, con una recta de tendencia **por continente**, títulos en español y un tema limpio:

R Code

[↺ Start Over](#)[▶ Run Code](#)

```
1 library(gapminder)
2 library(ggplot2)
3 library(dplyr)
4
5 gap_2007 <- gapminder |> filter(year == 2007)
6
7 ggplot(gap_2007, aes(x = gdpPercap, y = lifeExp,
8                      color = ____)) +
9   geom_point(alpha = 0.7, size = 2.5) +
10  geom_smooth(se = FALSE, method = "____") +
11  labs(
12    title = "____",
13    x = "PIB per cápita (USD)",
14    y = "Esperanza de vida (años)",
15    color = "Continente"
16  ) +
17  theme_minimal(base_size = 14)
```

Ejercicio Grupal (25-30 min)

Trabajarán en **salas pequeñas** con los datos de homicidios y pobreza preprocesados. Los detalles están en la **Guía de Ejercicio Grupal N° 3** disponible en el sitio del curso.



Recuerden

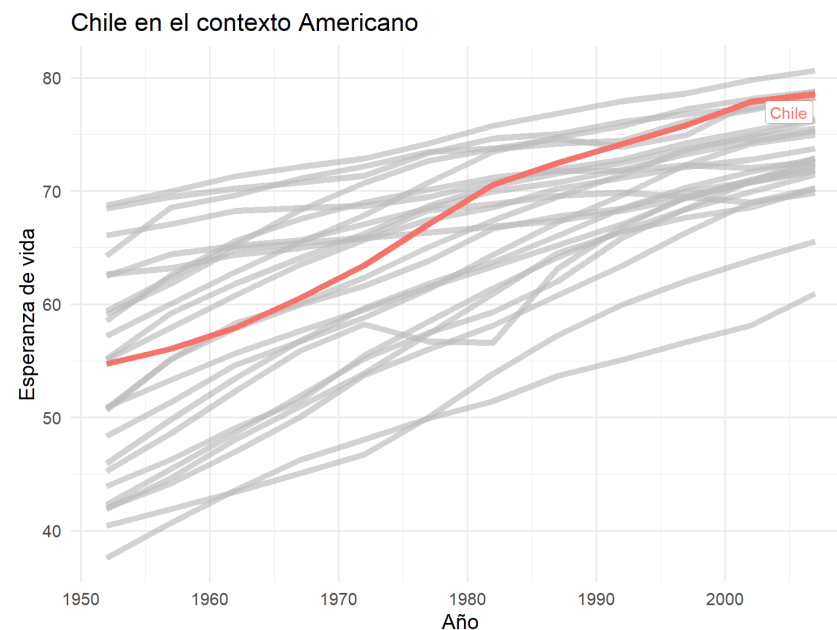
- ▶ Designen a una persona que comparta pantalla.
- ▶ Intenten resolver primero y luego revisen la pauta.
- ▶ El profesor y ayudante pasarán por las salas.

Extensiones

Resaltar objetos: `gghighlight`

Cuando hay **muchas categorías** en un gráfico, una leyenda cada elemento desvía la atención del gráfico (recordar el error #3). Una solución directa es usar `gghighlight()` para resaltar solo lo que nos importa comunicar. Por ejemplo, veamos la evolución de la esperanza de vida en Chile dentro del continente Americano

```
1 library(gghighlight)
2 ggplot(gapminder |> filter(continent == "A
3   geom_line(aes(year, lifeExp, color = cou
4   gghighlight(country == "Chile") +
5   labs(x = "Año", y = "Esperanza de vida",
6       title = "Chile en el contexto Ameri
7   theme_minimal(base_size = 14)
```



Resaltar con múltiples condiciones

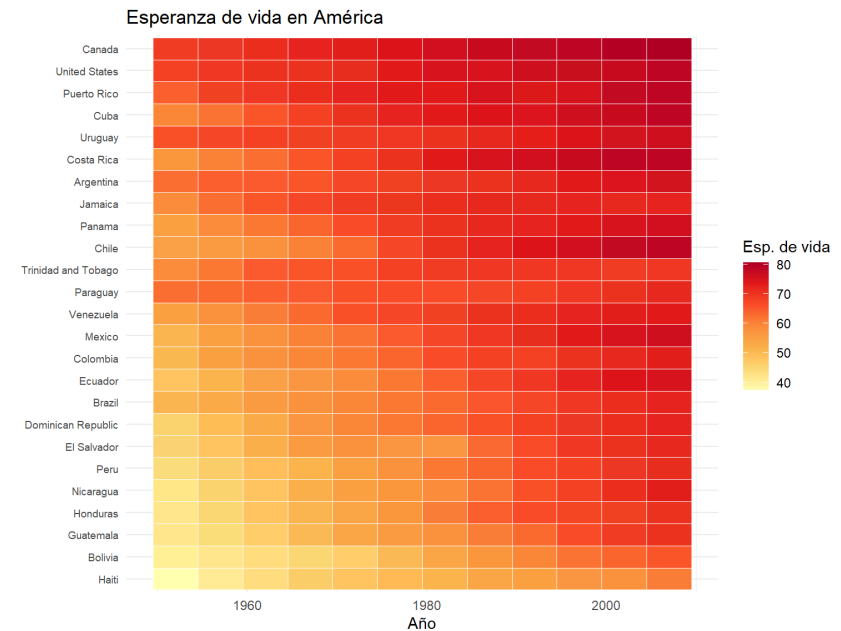
Heatmaps: `geom_tile()`

Un *heatmap* muestra la intensidad de una variable en una cuadrícula de dos categóricas y se puede crear mediante `geom_tile()`

```

1 gapminder |>
2   filter(continent == "Americas") |>
3   ggplot(aes(x = year,
4             y = reorder(country, lifeExp),
5             fill = lifeExp)) +
6   geom_tile(color = "white",
7             linewidth = 0.2) +
8   scale_fill_distiller(
9     palette = "YlOrRd",
10    direction = 1) +
11   labs(x = "Año",
12        y = "",
13        fill = "Esp. de vida",
14        title = "Esperanza de vida en Améri
15   theme_minimal(base_size = 11) +
16   theme(axis.text.y = element_text(size =

```





Pregunta N° 3

Tenemos un gráfico de línea con la evolución de la esperanza de vida de **25 países de América** y queremos destacar solo Chile. ¿Cuál es la mejor estrategia?

¿Qué haría?

Asignar un color distinto a cada país con `aes(color = country)`

Filtrar solo Chile con `filter(country == 'Chile')` y graficar

Usar `gghighlight(country == 'Chile')` para resaltar Chile y dejar el resto en gris

Usar `facet_wrap(vars(country))` para separar cada país

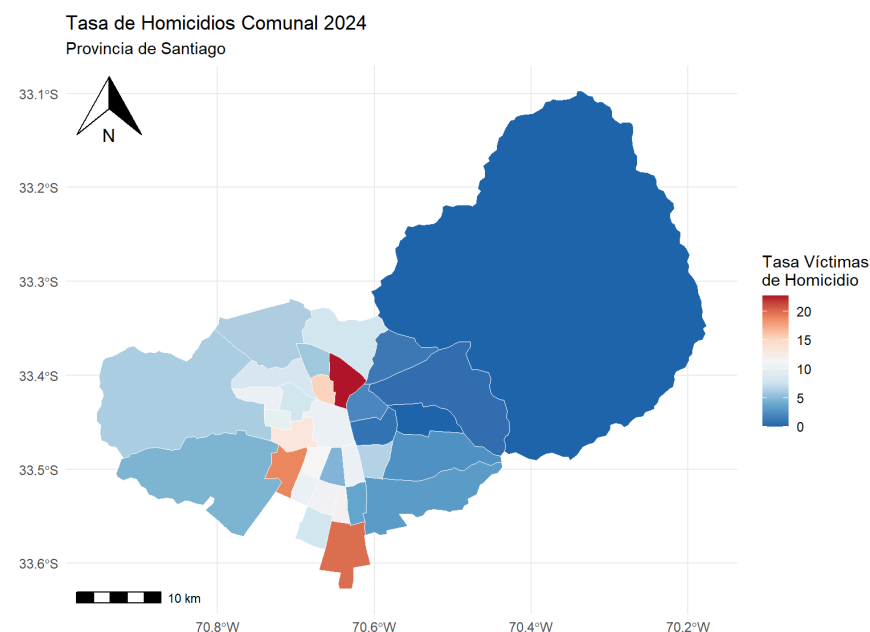
Revisar

Reiniciar

Mapas geográficos: `geom_sf()`

Para datos espaciales, `geom_sf()` dibuja polígonos desde un objeto `sf`. Si se combina con `scale_fill_distiller()`, es posible dibujar mapas temáticos coropléticos (*choropleth maps*) que pintan regiones geográficas en función de una variable numérica:

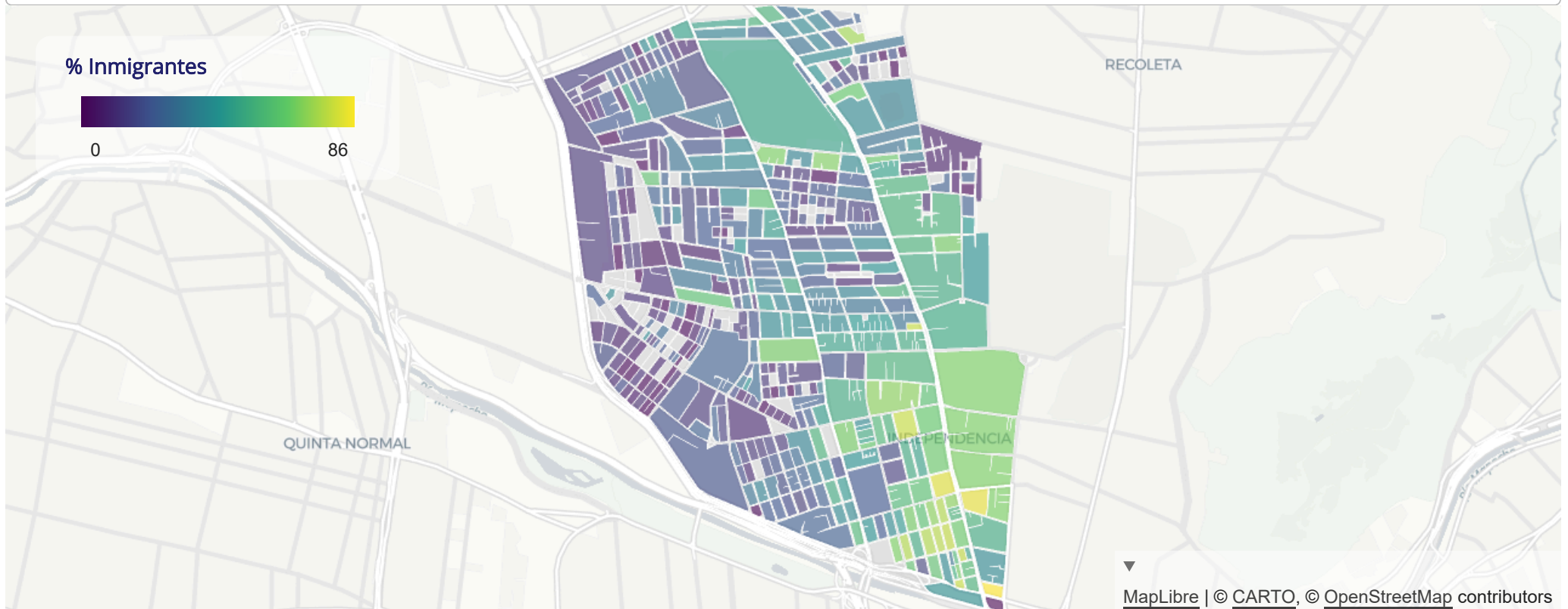
```
1 homicidios_rm_sf <- read_rds("data/clean/ho
2
3 ggplot(homicidios_rm_sf |> filter(provincia
4   geom_sf(aes(fill = tasa_hom_2024), color
5     linewidth = 0.05) +
6   scale_fill_distiller(palette = "RdBu", di
7   labs(title = "Tasa de Homicidios Comunal
8     subtitle = "Provincia de Santiago",
9     fill = str_wrap("Tasa Víctimas\nde H
10  annotation_scale() + #Escala del mapa
11  annotation_north_arrow(location = "tl") +
12  theme_minimal()
```



Mapas Interactivos

Con la librería `mapgl` es posible realizar mapas interactivos modernos y con una sintaxis moderna (también existen otros paquetes como `leaflet` o `mapview`)

```
1 library(mapgl)
2 manzanas_ind <- read_rds('data/clean/manzanas_independencia.rds') |>
3   select(comuna, manzent, n_per, n_inmigrantes, pct_inm_num) |>
4   rename(`% Inmigrantes` = pct_inm_num)
5
6 maplibre_view(manzanas_ind, column = "% Inmigrantes")
```



Resumen y Recomendaciones

1. Recuerden que los 3 elementos básicos para mostrar un gráfico en ggplot2 son:
 1. Datos (`data`): `ggplot(data)`
 2. Aesthetics (`aes`): `aes(x = variable_x, y= variable_y`
 3. Objeto Geométrico (`geom_*`): Por ejemplo `geom_line()`
2. La lógica de ggplot2 es ir **añadiendo capas** a la estructura definida previamente. Para añadir capas usamos `+` **no** el pipe `|>`
3. Al momento de construir una visualización siempre piensen en la historia que quieren contar (o el **mensaje** que quieren transmitir) y quién es su **audiencia**. A veces *menos es más* o lo *simple* es más efectivo que lo *complejo*.
4. [Awesome ggplot2](#) resume más de 100 paquetes relacionados con `ggplot2`. En función del objetivo que tengan, pueden encontrar paquetes que facilitan la visualización de información o ejecución de análisis específicos.

Y muchos recursos más

- ▶ [The R Graph Gallery](#).
- ▶ [Modern Data Visualization with R](#)
- ▶ [R CHARTS](#)
- ▶ Publicaciones de [#TidyTuesday](#) en donde la comunidad aporta soluciones a desafíos semanales con diversos datasets.

Recapitulación

¿Qué aprendimos hoy?

Sección	Contenido y Funciones
Principios	Explorar vs comunicar, Anscombe, errores de proporcionalidad/contexto/exceso de info
Grammar of graphics	<code>data</code> → <code>aes()</code> → <code>geom_*()</code> → <code>scale_*()</code> → <code>theme_*()</code>
Geoms	<code>geom_bar()</code> , <code>geom_histogram()</code> , <code>geom_density()</code> , <code>geom_boxplot()</code> , <code>geom_col()</code> , <code>geom_line()</code> , <code>geom_point()</code>
Control	<code>labs()</code> , <code>scale_x/y_continuous()</code> , <code>scale_color_brewer()</code> , <code>facet_wrap()</code>
Extensiones	<code>gghighlight()</code> , <code>geom_tile()</code> (heatmap), <code>geom_sf()</code> (mapas)
Exportar	<code>ggsave()</code>

La próxima clase aprenderemos a integrar estas visualizaciones, tablas y análisis en **documentos reproducibles** usando Quarto.

Bibliografía

- NUSSBAUMER KNAFLIC, COLE. 2025. *STORYTELLING WITH DATA: a data visualization guide for business professionals*. S.I.: JOHN WILEY.
- Tufte, Edward R. 2001. *The Visual Display of Quantitative Information, 2nd Ed.* Cheshire, Conn.
- Tukey, John Wilder. 1977. *Exploratory data analysis*. Addison-Wesley series en behavioral science. Reading, Mass: Addison-Wesley Pub. Co.
- Wickham, Hadley. 2015. «ggplot2: Elegant Graphics for Data Analysis (3e)». <https://ggplot2-book.org/>.
- Wilke, C. 2019. *Fundamentals of data visualization: a primer on making informative and compelling figures*. First edition. Sebastopol, CA: O'Reilly Media.